



||JAI SRI GURUDEV||
Sri AdichunchanagiriShikshana Trust ®
SJB INSTITUTE OF TECHNOLOGY
(A Constituent Institution of BGS & SJB Group of Institution & Hospitals)
No. 67, BGS Health & Education City, Dr. Vishnuvardhan Road, Kengeri, Bengaluru - 560 060



Department of Computer Science & Engineering



Subject: Theory of Computation
Subject Code: 23CSI602

Module 3: Context-Free Grammars

1. Introduction to Rewrite Systems and Grammars

What is Rewrite System?

A rewrite system (or production system or rule-based system) is a list of rules, and an algorithm for applying them. Each rule has a left-hand side and a right-hand side.

$$\begin{array}{c} X \rightarrow Y \\ \text{(LHS)} \quad \text{(RHS)} \end{array}$$

Examples of rewrite-system rules: $S \rightarrow aSb$, $aS \rightarrow \epsilon$, $aSb \rightarrow bSabSa$

When a rewrite system R is invoked on some initial string w , it operates as follows: $\text{simple-rewrite}(R; \text{rewrite system}, w; \text{initial string}) =$

1. Set working-string to w .
2. Until told by R to halt do:
Match the LHS of some rule against some part of working-string.
Replace the matched part of working-string with the RHS of the rule that was matched.
3. Return working-string.

If it returns some string then R can derive from w or there exists a derivation in R of w .

Examples:

1. A rule is simply a pair of strings where, if the string on the LHS matches, it is replaced by the string on the RHS.
2. The rule $axa \rightarrow a$ squeezes out whatever comes between a pair of a 's.
3. The rule $ab^*ab^*a \rightarrow a$ squeezes out b 's between a 's.

Rewrite systems can be used to define functions. We write rules that operate on an input string to produce the required output string. Rewrite systems can be used to define languages. The rewrite system that is used to define a language is called a grammar.

Grammars Define Languages

A grammar is a set of rules that are stated in terms of two alphabets:

- a terminal alphabet, Σ , that contains the symbols that make up the strings in $L(G)$,
- a nonterminal alphabet, the elements of which will function as working symbols that will be used while the grammar is operating. These symbols will disappear by the time the grammar finishes its job and generates a string.
- A grammar has a unique start symbol, often called S .

A rewrite system formalism specifies:

- The form of the rules that are allowed.
- The algorithm by which they will be applied.
- How its rules will be applied?

Using a Grammar to Derive a String

Simple-rewrite (G, S) will generate the strings in L(G). The symbol $\Rightarrow$ to indicate steps in a derivation.

Given: $S \rightarrow aS$ ----- rule1

$S \rightarrow \epsilon$ ----- rule2

A derivation could begin with: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$

Generating Many Strings

LHS of Multiple rules may match with the working string.

Given: $S \rightarrow aSb$ ----- rule 1

$S \rightarrow bSa$ ----- rule2

$S \rightarrow \epsilon$ ----- rule3

Derivation so far: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow$

There are three choices at the next step:

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb$ (using rule1),

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabSabb$ (using rule2),

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$ (using rule3).

One rule may match in more than one way.

Given: $S \rightarrow aTTb$ ----- rule 1

$T \rightarrow bTa$ ----- rule2

$T \rightarrow \epsilon$ ----- rule3

Derivation so far: $S \Rightarrow aTTb \Rightarrow$

Two choices at the next step:

$S \Rightarrow aTTb \Rightarrow abTaTb \Rightarrow$

$S \Rightarrow aTTb \Rightarrow aTbTab \Rightarrow$

When to Stop

Case 1: The working string no longer contains any nonterminal symbols (including, when it is ϵ). In this case, we say that the working string is generated by the grammar. Example: S

$\Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$

Case 2: There are nonterminal symbols in the working string but none of them appears on the left-hand side of any rule in the grammar. In this case, we have a blocked or non-terminated derivation but no generated string.

Given: $S \rightarrow aSb$ ----- rule1

$S \rightarrow bTa$ ----- rule2

$S \rightarrow \epsilon$ ----- rule3

Derivation so far: $S \Rightarrow aSb \Rightarrow abTab \Rightarrow$

Case 3: It is possible that neither case 1 nor case 2 is achieved. Given:

$S \rightarrow Ba$ ----- rule 1

$B \rightarrow bB$ ----- rule2

Then all derivations proceed as: $S \Rightarrow Ba \Rightarrow bBa \Rightarrow bbBa \Rightarrow bbbBa \Rightarrow bbbbBa \Rightarrow \dots$. The grammar generates the language $\emptyset$.

2. Context-Free Grammar and Languages

Recall Regular Grammar which has a left-hand side that is a single nonterminal and have a right-hand side that is ϵ or a single terminal or a single terminal followed by a single nonterminal.

$$X \rightarrow Y$$

(NT) (ϵ or T or T NT)

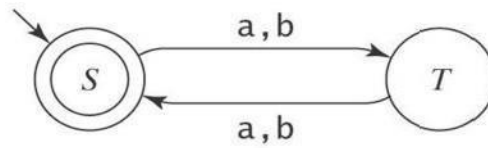
Example: $L = \{w \in \{a,b\}^* : |w| \text{ is even}\}$

RE = $((aa)(ab)(ba)(bb))^*$

G:

- $S \rightarrow \epsilon$
- $S \rightarrow aT$
- $S \rightarrow bT$
- $T \rightarrow aS$
- $T \rightarrow bS$

M:



Context Free Grammars

$$X \rightarrow Y$$

(NT)(No restriction)

No restrictions on the form of the right hand sides. But require single non-terminal on left hand side.

Example: $S \rightarrow \epsilon, S \rightarrow a, S \rightarrow T, S \rightarrow aSb, S \rightarrow aSbbT$ are allowed.

$ST \rightarrow aSb, a \rightarrow aSb, \epsilon \rightarrow aa$ are not allowed.

The name for these grammars “Context Free” makes sense because using these rules the decision to replace a nonterminal by some other sequence is made without looking at the context in which the nonterminals occurs.

Definition Context-Free Grammar

A context-free grammar G is a quadruple (V, Σ, R, S) , where:

- V is the rule alphabet, which contains nonterminals and terminals.
- Σ (the set of terminals) is a subset of V ,
- R (the set of rules) is a finite subset of $(V - \Sigma) \times V^*$,
- S (the start symbol) is an element of $V - \Sigma$.

Given a grammar G , define $x \Rightarrow_G y$ to be the binary relation derives-in-one-step, defined so that $\forall x, y \in V^*$ ($x \Rightarrow_G y$ iff $x = \alpha A \beta, y = \alpha \gamma \beta$ and there exists a rule $A \rightarrow \gamma$ is in R_G)

Any sequence of the form $w_0 \Rightarrow_G w_1 \Rightarrow_G w_2 \Rightarrow_G \dots \Rightarrow_G w_n$ is called a derivation in G . Let $\Rightarrow_G^*$ be the reflexive, transitive closure of $\Rightarrow_G$. We'll call $\Rightarrow_G^*$ the derive relation.

A derivation will halt whenever no rules on the left hand side matches against working-string. At every step, any rule that matches may be chosen.

Language generated by G , denoted $L(G)$, is: $L(G) = \{w \in \Sigma^* : S \Rightarrow_G^* w\}$. A language L is context-free iff it is generated by some context-free grammar G . The context-free languages (or CFLs) are a proper superset of the regular languages.

Example: $L = A^n B^n = \{a^n b^n : n \geq 0\} = \{\epsilon, ab, aabb, aaabbb, \dots\}$ $G = \{\{S, a, b\}, \{a, b\}, R, S\}$, where: $R = \{S \rightarrow aSb, S \rightarrow \epsilon\}$

Example derivation in G : $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaasbbb \Rightarrow aaabbb$ or $S \Rightarrow^* aaabbb$

Recursive Grammar Rules

A grammar is recursive iff it contains at least one recursive rule. A rule is recursive iff it is X

$\rightarrow w_1 Y w_2$, where: $Y \Rightarrow^* w_3 X w_4$ for some w_1, w_2, w_3 , and w_4 in V^* . Expanding a non-terminal according to these rules can eventually lead to a string that includes the same non-terminal again.

Example 1: $L = A^n B^n = \{a^n b^n : n \geq 0\}$ Let $G = (\{S, a, b\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow \epsilon\}, S)$

Example 2: Regular grammar whose rules are $\{S \rightarrow aT, T \rightarrow aW, W \rightarrow aS, W \rightarrow a\}$ Example 3:

The Balanced Parenthesis language

$Bal = \{w \in \{(), \{, \}^* : \text{the parentheses are balanced}\} = \{\epsilon, (), (()), (()), (()) \dots\}$

$G = \{\{S, (), \{, \}, \{, \}, R, S\}$ where $R = \{S \rightarrow \epsilon, S \rightarrow SSS \rightarrow (S)\}$

Some example derivations in G :

$S \Rightarrow (S) \Rightarrow ()$

$S \Rightarrow (S) \Rightarrow (SS) \Rightarrow ((S)S) \Rightarrow ((()S)) \Rightarrow (((()S))) \Rightarrow (((()())))$

So, $S \Rightarrow^* ()$ and $S \Rightarrow^* (()())$

Recursive rules make it possible for a finite grammar to generate an infinite set of strings.

Self-Embedding Grammar Rules

A grammar is self-embedding iff it contains at least one self-embedding rule. A rule in a grammar G is self-embedding iff it is of the form $X \rightarrow w_1 Y w_2$, where $Y \Rightarrow^* w_3 X w_4$ and both $w_1 w_3$ and $w_4 w_2$ are in Σ^+ . No regular grammar can impose such a requirement on its strings.

Example: $S \rightarrow aSa$ is self-embedding

$S \rightarrow aS$ is recursive but not self-embedding

$S \rightarrow aT$

$T \rightarrow Sa$ is self-embedding

Example: $\text{PalEven} = \{ww^R : w \in \{a,b\}^*\}$ = The set of even length palindromes of a 's and b 's. $L = \{\epsilon, aa, bb, aaaa, abba, baab, bbbb, \dots\}$

$G = \{ \{S, a, b\}, \{a, b\}, R, S \}$, where:

$R = \{ S \rightarrow aSa \text{ ----- rule1}$

$S \rightarrow bSb \text{ ----- rule2}$

$S \rightarrow \epsilon \text{ ----- rule3 } \}$.

Example derivation in G :

$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$

Where Context-Free Grammars Get Their Power

If a grammar G is not self-embedding then $L(G)$ is regular. If a language L has the property that every grammar that defines it is self-embedding, then L is not regular.

More flexible grammar-writing notations

- a. Notation for writing practical context-free grammars. The symbol $|$ should be read as "or". It allows two or more rules to be collapsed into one.

Example:

$S \rightarrow aSb$

$S \rightarrow bSa \quad S \rightarrow \epsilon$ can be written as $S \rightarrow aSb / bSa / \epsilon$

$\rightarrow \epsilon$

- b. Allow a nonterminal symbol to be any sequence of characters surrounded by angle brackets.

Example 1: BNF for a Java Fragment

$\langle \text{block} \rangle ::= \{ \langle \text{stmt-list} \rangle \} \{ \}$

$\langle \text{stmt-list} \rangle ::= \langle \text{stmt} \rangle | \langle \text{stmt-list} \rangle \langle \text{stmt} \rangle$

$\langle \text{stmt} \rangle ::= \langle \text{block} \rangle | \text{while}(\langle \text{cond} \rangle) \langle \text{stmt} \rangle | \text{if}$

$(\langle \text{cond} \rangle) \langle \text{stmt} \rangle |$

$\text{do} \langle \text{stmt} \rangle \text{while}(\langle \text{cond} \rangle); |$

$\langle \text{assignment-stmt} \rangle; |$

$\text{return} | \text{return} \langle \text{expression} \rangle |$

$\langle \text{method-invocation} \rangle;$

Example2: ACFG for C++ compound statements:

```
<compoundstmt> → { <stmt list> }  
<stmtlist> → <stmt> <stmt list> | epsilon  
<stmt> → <compoundstmt>  
<stmt> → if(<expr>) <stmt>  
<stmt> → if(<expr>) <stmt> else <stmt>  
<stmt> → while(<expr>) <stmt>  
<stmt> → do <stmt> while(<expr>);  
<stmt> → for( <stmt> <expr>; <expr>) <stmt>  
<stmt> → case <expr>: <stmt>  
<stmt> → switch(<expr>) <stmt>  
<stmt> → break ; | continue;  
<stmt> → return <expr>; | goto <id>;
```

Example3: A Fragment of English Grammar

Notational conventions used are

- Nonterminal = whose first symbol is an uppercase letter
- NP = derive noun phrase
- VP = derive verb phrase

$S \rightarrow NP VP$

$NP \rightarrow \text{the Nominal} \mid \text{a Nominal} \mid \text{Nominal} \mid \text{Proper Noun} \mid \text{NPPP Nominal} \rightarrow$

$N \mid \text{Adjs } N$

$N \rightarrow \text{cat} \mid \text{dogs} \mid \text{bear} \mid \text{girl} \mid \text{chocolate} \mid \text{rifle} \mid \text{Proper Noun}$

$\rightarrow \text{Chris} \mid \text{Fluffy}$

$\text{Adjs} \rightarrow \text{Adj Adjs} \mid \text{Adj}$

$\text{Adj} \rightarrow \text{young} \mid \text{older} \mid \text{smart} \mid \text{VP}$

$\rightarrow V \mid V NP \mid VP PP$

$V \rightarrow \text{like} \mid \text{likes} \mid \text{thinks} \mid \text{shots} \mid \text{smells} \mid PP \rightarrow$

$\text{Prep } NP$

$\text{Prep} \rightarrow \text{with}$

3. Designing Context-Free Grammars

If L has a property that every string in it has two regions & those regions must bear some relationship to each other, then the two regions must be generated in tandem. Otherwise, there is no way to enforce the necessary constraint.

Example1: $L = \{a^m b^n c^m : n, m \geq 0\} = L = \{\epsilon, ab, c, abc, abcc, aabbc, \dots\}$

The c^m portion of any string in L is completely independent of the $a^n b^n$ portion, so we should generate the two portions separately and concatenate them together.

$G = (\{S, A, C, a, b, c\}, \{a, b, c\}, R, S)$ where:

$$\begin{aligned} R = \{ & S \rightarrow AC && /* generate the two independent portions \\ & A \rightarrow aAb \mid \epsilon && /* generate the $a^n b^n$ portion, from the outside in \\ & C \rightarrow cC \mid \epsilon && /* generate the c^m portion \end{aligned}$$

Example derivation in G for string $abcc$:

$S \Rightarrow AC \Rightarrow aAbC \Rightarrow abC \Rightarrow abcC \Rightarrow abccC \Rightarrow abcc$

Example2: $L = \{a^i b^j c^k : j = i + k, i, k \geq 0\}$ on substituting $j = i + k \Rightarrow L = \{a^i b^i b^k c^k : i, k \geq 0\}$ $L = \{\epsilon, abbc, aabbbbcc, abbbcc, \dots\}$

The $a^i b^i$ portion of any string in L is completely independent of the $b^k c^k$ portion, so we should generate the two portions separately and concatenate them together.

$G = (\{S, A, B, a, b, c\}, \{a, b, c\}, R, S)$ where:

$$\begin{aligned} R = \{ & S \rightarrow AB && /* generate the two independent portions \\ & A \rightarrow aAb \mid \epsilon && /* generate the $a^i b^i$ portion, from the outside in \\ & B \rightarrow bBc \mid \epsilon && /* generate the $b^k c^k$ portion \end{aligned}$$

Example derivation in G for string $abbc$:

$S \Rightarrow AB \Rightarrow aAbB \Rightarrow abB \Rightarrow abbBc \Rightarrow abbc$

Example3: $L = \{a^i b^j c^k : i = j + k, j, k \geq 0\}$ on substituting $i = j + k \Rightarrow L = \{a^k a^j b^j c^k : j, k \geq 0\}$ $L = \{\epsilon, ac, ab, aabc, aaabcc, \dots\}$

The $a^i b^j$ is the inner portion and $a^k c^k$ is the outer portion of any string in L . $G = (\{S, A, a, b, c\}, \{a, b, c\}, R, S)$ where:

$$\begin{aligned} R = \{ & S \rightarrow aSc \mid A && /* generate the $a^k c^k$ outer portion \\ & A \rightarrow aAb \mid \epsilon && /* generate the $a^j b^j$ inner portion \end{aligned}$$

Example derivation in G for string $aabc$:

$S \Rightarrow aSc \Rightarrow aAc \Rightarrow aaAbc \Rightarrow aabc$

Example4: $L = \{a^n w w^R b^n : w \in \{a, b\}^*\} = \{\epsilon, ab, aaab, abbb, aabbab, aabbbbab, \dots\}$

The $a^n b^n$ is the inner portion and ww^R is the outer portion of any string in L . $G = (\{S, A, a, b\}, \{a, b\}, R, S)$, where:

$$\begin{aligned} R = \{ & S \rightarrow aSb && \text{rule 1} \\ & S \rightarrow A && \text{rule 2} \\ & A \rightarrow aAa && \text{rule 3} \\ & A \rightarrow bAb && \text{rule 4} \\ & A \rightarrow \epsilon && \text{rule 5} \end{aligned} \}$$

Example derivation in G for string $aabbab$:

$S \Rightarrow aSb \Rightarrow aAb \Rightarrow aaAab \Rightarrow aabAbab \Rightarrow aabbab$

Example5:EqualNumbersofa'sandb's. $= \{w \in \{a,b\}^* : \#_a(w) = \#_b(w)\}$. $L = \{\epsilon, ab, ba, abba, aabb, baba, bbaa, \dots\}$

$G = \{ \{S, a, b\}, \{a, b\}, R, S \}$, where:

$R = \{ \begin{array}{l} S \rightarrow aSb \text{ ----- rule1} \\ S \rightarrow bSa \text{ ----- rule2} \\ S \rightarrow SS \text{ ----- rule3} \\ S \rightarrow \epsilon \text{ ----- rule4} \end{array} \}$.

Example derivation in G for string $abba$: S

$\Rightarrow aSa \Rightarrow abSba \Rightarrow abba$

Example6

$L = \{a^i b^j : 2i = 3j + 1\} = \{a^2 b^1, a^5 b^3, a^8 b^5, \dots\}$

$G = \{ \{S, a, b\}, \{a, b\}, R, S \}$, where:

$a^i b^j$	$2i = 3j + 1$
$a^2 b^1$	$2*2 = 3*1 + 1 = 4$
$a^5 b^3$	$2*5 = 3*3 + 1 = 10$
$a^8 b^5$	$2*8 = 3*5 + 1 = 16$

$R = \{S \rightarrow aaaSbb \mid aab\}$

Example derivation in G for string $aaaaabbb$:

$S \Rightarrow aaaSbb \Rightarrow aaaaaabbb$

4. Simplifying Context-Free Grammars

Two algorithms used to simplify CFG

- a. To find and remove unproductive variables using `removeunproductive(G:CFG)`
- b. To find and remove unreachable variables using `removeunreachable(G:CFG)`

- a. Removing Unproductive Nonterminals:

`Removeunproductive (G: CFG) =`

1. $G' = G$.
2. Mark every nonterminal symbol in G' as unproductive.
3. Mark every terminal symbol in G' as productive.
4. Until one entire pass has been made without any new symbol being marked do: For each rule $X \rightarrow \alpha$ in R do:

If every symbol in α has been marked as productive and X has not yet been marked as productive then:
 Mark X as productive.
5. Remove from G' every unproductive symbol.
6. Remove from G' every rule that contains an unproductive symbol.
7. Return G' .

Example: $G = (\{S, A, B, C, D, a, b\}, \{a, b\}, R, S)$, where $R = \{$

$S \rightarrow AB \mid AC$

$A \rightarrow aAb \mid \varepsilon$

$B \rightarrow bA$

$C \rightarrow bCa$

$D \rightarrow AB\}$

1) a and b terminal symbols are productive

2) A is productive (because $A \rightarrow aAb$)

3) B is productive (because $B \rightarrow bA$)

4) S & D are productive (because $S \rightarrow AB$ & $D \rightarrow AB$)

5) C is unproductive

On eliminating C from both LHS and RHS the rule set R' obtained is $R' = \{$

$S \rightarrow AB \quad A \rightarrow aAb \mid \varepsilon \quad B \rightarrow bA \quad D \rightarrow AB\}$

b. Removing Unreachable Nonterminals

Remove unreachable (G : CFG) =

1. $G' = G$.

2. Mark S as reachable.

3. Mark every other nonterminal symbol as unreachable.

4. Until one entire pass has been made without any new symbol being marked do: For each rule $X \rightarrow \alpha A \beta$ (where $A \in V - \Sigma$) in R do:

If X has been marked as reachable and A has not then: Mark A as reachable.

5. Remove from G' every unreachable symbol.

6. Remove from G' every rule with an unreachable symbol on the left-hand side.

7. Return G' .

Example

$G = (\{S, A, B, C, D, a, b\}, \{a, b\}, R, S)$, where $R' = \{S$

$\rightarrow AB$

$A \rightarrow aAb \mid \varepsilon$

$B \rightarrow bA$

$D \rightarrow AB\}$

S, A, B are reachable but D is not reachable, on eliminating D from both LHS and RHS the rule set R'' is

$R'' = \{S \rightarrow AB$

$A \rightarrow aAb \mid \varepsilon$

$B \rightarrow bA\}$

5. Proving the Correctness of a Grammar

Given some language L and a grammar G , can we prove that G is correct (i.e. it generates exactly the strings in L)

To do so, we need to prove two things:

1. Prove that G generates only strings in L .
2. Prove that G generates all the strings in L .

6. Derivations and Parse Trees

Algorithms used for generation and recognition must be systematic. The expansion order is important for algorithms that work with CFG. To make it easier to describe such algorithms, we define two useful families of derivations.

- a. A leftmost derivation is one in which, at each step, the leftmost nonterminal in the working string is chosen for expansion.
- b. A rightmost derivation is one in which, at each step, the rightmost nonterminal in the working string is chosen for expansion.

Example 1: $S \rightarrow AB \mid aAB \rightarrow b$

Left-most derivation for string aab is $S \Rightarrow AB \Rightarrow AaB \Rightarrow aaB \Rightarrow aab$

Right-most derivation for string aab is $S \Rightarrow AB \Rightarrow Ab \Rightarrow Aab \Rightarrow aab$

Example 2: $S \rightarrow iCtS \mid iCtSeS \mid x \quad C \rightarrow y$

Left-most Derivation for string $iytiytxex$ is $S \Rightarrow iCtS \Rightarrow iytS \Rightarrow iytiCtSeS \Rightarrow iytiySeS \Rightarrow iytiytxe \Rightarrow iytiytxex$

Right-most Derivation for string $iytiytxex$ is $S \Rightarrow iCtSeS \Rightarrow iCtSex \Rightarrow iCtiCtSex \Rightarrow iCtiCtxex \Rightarrow iCtiytxex \Rightarrow iytiytxex$

Example 3: A Fragment of English Grammar are $S \rightarrow$

$NP VP$

$NP \rightarrow the Nominal \mid a Nominal \mid Nominal \mid Proper Noun \mid NPPP Nominal$

$\rightarrow N \mid Adjs N$

$N \rightarrow cat \mid dogs \mid bear \mid girl \mid chocolate \mid rifle$

$Proper Noun \rightarrow Chris \mid Fluffy$

$Adjs \rightarrow Adj Adjs \mid Adj$

$Adj \rightarrow young \mid older \mid smart$

$VP \rightarrow V \mid V NP \mid VP PP$

$V \rightarrow like \mid likes \mid thinks \mid shots \mid smells$

$PP \rightarrow$

$Prep NP$

$Prep \rightarrow with$

Left-most Derivation for the string "the smart cat smells chocolate" $S \Rightarrow$

NP VP

$\Rightarrow$ the Nominal VP
 $\Rightarrow$ the Adjs NVP
 $\Rightarrow$ the Adj NVP
 $\Rightarrow$ the smart NVP
 $\Rightarrow$ the smart cat VP
 $\Rightarrow$ the smart cat V NP
 $\Rightarrow$ the smart cat smells NP
 $\Rightarrow$ the smart cat smells Nominal
 $\Rightarrow$ the smart cat smells N
 $\Rightarrow$ the smart cat smells chocolate

Right-most Derivation for the string "the smart cat smells chocolate" $S \Rightarrow$

NP VP

$\Rightarrow$ NP VNP
 $\Rightarrow$ NP VNominal
 $\Rightarrow$ NP VN
 $\Rightarrow$ NP Vchocolate
 $\Rightarrow$ NP smells chocolate
 $\Rightarrow$ the Nominal smells chocolate
 $\Rightarrow$ the Adjs N smells chocolate
 $\Rightarrow$ the Adjscat smells chocolate
 $\Rightarrow$ the Adjcat smells chocolate
 $\Rightarrow$ the smart cat smells chocolate

Parse Trees

Regular grammar: in most applications, we just want to describe the set of strings in a language.

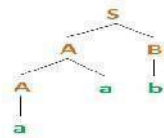
Context-free grammar: we also want to assign meaning to the strings in a language, for which we care about internal structure of the strings. Parse trees capture the essential grammatical structure of a string. A program that produces such trees is called a parser. A parse tree is an (ordered, rooted) tree that represents the syntactic structure of a string according to some formal grammar. In a parse tree, the interior nodes are labeled by nonterminals of the grammar, while the leaf nodes are labeled by terminals of the grammar or ϵ .

A parse tree, derived by a grammar $G = (V, S, R, S)$, is a rooted, ordered tree in which:

1. Every leaf node is labeled with an element of $\Sigma \cup \{ \epsilon \}$,
2. The root node is labeled S ,
3. Every other node is labeled with some element of: $V - \Sigma$, and
4. If m is a non-leaf node labeled X and the children of m are labeled $x_1, x_2, \dots, x_n$, then R contains the rule $X \rightarrow x_1, x_2, \dots, x_n$.

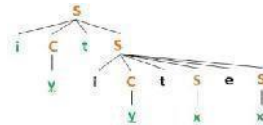
Example1: $S \rightarrow AB \mid aaB$ $A \rightarrow a \mid Aa$ $B \rightarrow b$

Left-most derivation for the string aab is $S \Rightarrow AB \Rightarrow AaB \Rightarrow aaB \Rightarrow aab$ Parse tree obtained is

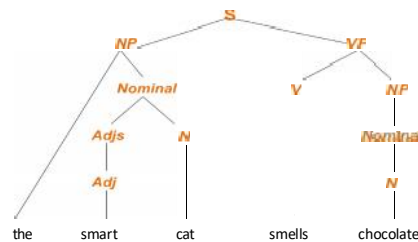


Example2: $S \rightarrow iCtS \mid iCtSeS \mid x$ $C \rightarrow y$

Left-most Derivation for string $iytiytxexis$ is $S \Rightarrow iCtS \Rightarrow iytS \Rightarrow iytiCtSeS \Rightarrow iytiytxeS \Rightarrow iytiytxex$



Example 3: Parse Tree -Structure in English for the string “the smartcat smells chocolate”. It is clear from the tree that this sentence is not about cats smells or smartcat smells.



A parse tree may correspond to multiple derivations. From the parse tree, we cannot tell which of the following is used in derivation:

$S \Rightarrow NPVP \Rightarrow \text{the Nominal VP} \Rightarrow$

$S \Rightarrow NPVP \Rightarrow NPVNP \Rightarrow$

Parse trees capture the important structural facts about a derivation but throw away the details of the nonterminal expansion order. The order has no bearing on the structure we wish to assign to a string.

Generative Capacity

Because parse trees matter, it makes sense, given a grammar G , to distinguish between:

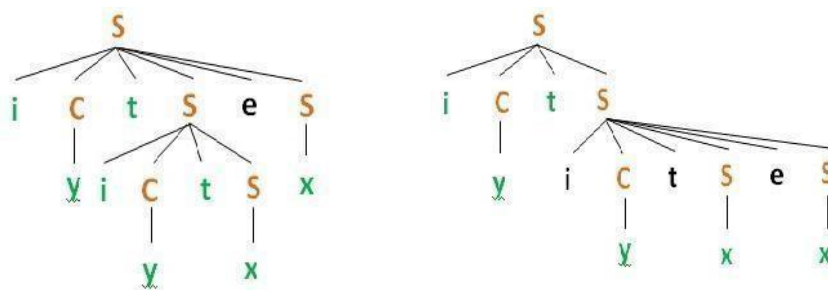
1. G 's weak generative capacity, defined to be the set of strings, $L(G)$, that G generates, and
2. G 's strong generative capacity, defined to be the set of parse trees that G generates.

When we design a grammar, it will be important that we consider both their weak and their strong generative capacities.

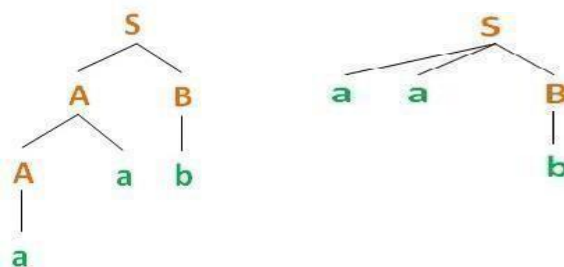
7. Ambiguity

Sometimes a grammar may produce more than one parse tree for some (or all) of the strings it generates. When this happens we say that the grammar is ambiguous. A grammar is ambiguous iff there is at least one string in $L(G)$ for which G produces more than one parse tree.

Left-most Derivation1 $\text{forthe string}(()()) \text{is } S \Rightarrow S \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow (())S \Rightarrow (())(S) \Rightarrow (())()$ Left-most Derivation2 $\text{forthe string}(()()) \text{is } S \Rightarrow SS \Rightarrow SSS \Rightarrow SS \Rightarrow (S)S \Rightarrow ((S))S \Rightarrow (())S \Rightarrow (())(S) \Rightarrow (())()$


$$\begin{aligned} \text{Right-most Derivation for the string } iytixex \text{ is } & S \Rightarrow iCtSeS \Rightarrow iCtSex \Rightarrow iCtiCtSex \\ & \Rightarrow iCtiCtxex \Rightarrow iCtiytxex \Rightarrow iytixex \end{aligned}$$


Right-most derivation for string aab is $S \Rightarrow aaB \Rightarrow aab$



65

Why Is Ambiguity a Problem?

With regular languages, for most applications, we do not care about assigning internal structure to strings.

With context-free languages, we usually do care about internal structure because, given a string w , we want to assign meaning to w . It is generally difficult, if not impossible, to assign a unique meaning without a unique parse tree. So an ambiguous G , which fails to produce a unique parse tree is a problem.

Example: Arithmetic Expressions G

$= (V, \Sigma, R, E)$, where

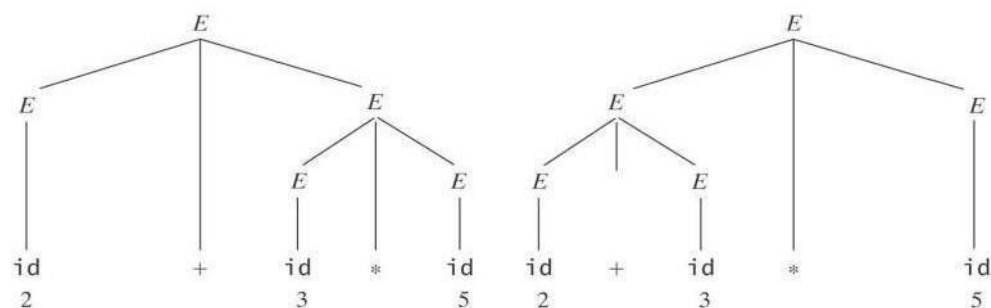
$V = \{+, *, (,), \text{id}, E\}$,

$\Sigma = \{+, *, (,), \text{id}\}$,

$R = \{E \rightarrow E + E, E \rightarrow E * E, E \rightarrow (E), E \rightarrow \text{id}\}$

Consider string $2+3*5$ written as $\text{id} + \text{id} * \text{id}$, left-most derivation for string $\text{id} + \text{id} * \text{id}$ is $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow \text{id} + E * E \Rightarrow \text{id} + \text{id} * E \Rightarrow \text{id} + \text{id} * \text{id}$.

Similarly the right-most derivation for string $\text{id} + \text{id} * \text{id}$ is $E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow E + E * \text{id} \Rightarrow E + \text{id} * \text{id} \Rightarrow \text{id} + \text{id} * \text{id}$. The parse trees obtained for both the derivations are:-



Should the evaluation of this expression return 17 or 25? Designers of practical languages must be careful that they create languages for which they can write unambiguous grammars. Techniques for Reducing Ambiguity

No general purpose algorithm exists to test for ambiguity in a grammar or to remove it when it is found. But we can reduce ambiguity by eliminating

- rules like $S \rightarrow \epsilon$
- Rules with symmetric right-hand sides
 - A grammar is ambiguous if it is both left and right recursive.
 - Fix: remove right recursion
 - $S \rightarrow SS$ or $E \rightarrow E + E$
- Rules that lead to ambiguous attachment of optional postfixes.

a. Eliminating ϵ -Rules

Let $G = (V, \Sigma, R, S)$ be a CFG. The following algorithm constructs a G' such that $L(G') = L(G) - \{\epsilon\}$ and G' contains no ϵ -rules:

removeEps(G :CFG)=

1. Let $G' = G$.
2. Find the set N of nullable variables in G' .
3. Repeat until G' contains no modifiable rules that haven't been processed:
Given the rule $P \rightarrow \alpha Q \beta$, where $Q \in N$, add the rule $P \rightarrow \alpha \beta$ if it is not already present and if $\alpha \beta \neq \epsilon$ and if $P \neq \alpha \beta$.
4. Delete from G' all rules of the form $X \rightarrow \epsilon$.
5. Return G' .

Nullable Variables & Modifiable Rules

A variable X is nullable iff either:

- (1) there is a rule $X \rightarrow \epsilon$, or
 - (2) there is a rule $X \rightarrow PQR \dots$ and $P, Q, R, \dots$ are all nullable.
- So compute N , the set of nullable variables, as follows:

Set N to the set of variables that satisfy (1).

Until a new iteration is made without adding anything to N do Evaluate all other variables with respect to (2).

If any variable satisfies (2) and is not in N , insert it.

A rule is modifiable iff it is of the form: $P \rightarrow \alpha Q \beta$, for some nullable Q . Example: $G = (\{S, T, A, B, C, a, b, c\}, \{a, b, c\}, R, S)$,

$R = \{S \rightarrow aTa \quad T \rightarrow ABC \quad A \rightarrow aA \mid C \quad B \rightarrow Bb \mid C \quad C \rightarrow c \mid \epsilon\}$

Applying removeEps

Step2: $N = \{C\}$

Step2.2 pass1: $N = \{A, B, C\}$

Step2.2 pass2: $N = \{A, B, C, T\}$

Step2.2 pass3: no new element found.

Step2: halts.

Step3: add the following new rules to G' .

$\{S \rightarrow aa$

$T \rightarrow AB \mid BC \mid AC \mid A \mid B \mid C$

$A \rightarrow a$

$B \rightarrow b \}$

The rules obtained after eliminating ϵ -rules:

$\{S \rightarrow aTa \mid aa$

$T \rightarrow ABC \mid AB \mid BC \mid AC \mid A \mid B \mid C$

$A \rightarrow aA \mid C \mid a$

$B \rightarrow Bb \mid C \mid b$

$C \rightarrow c\}$

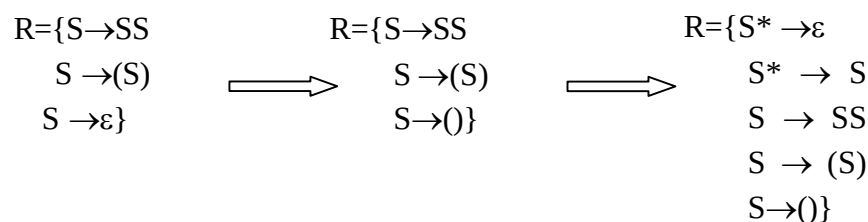
What if $\epsilon \in L$?

Sometimes $L(G)$ contains ϵ and it is important to retain it. To handle this case the algorithm used is

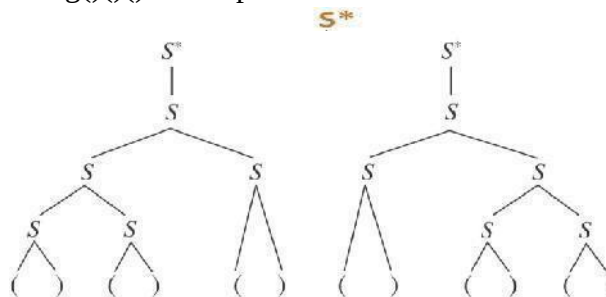
atmostoneEps($G:CFG$)=

1. $G' = \text{removeEps}(G)$.
2. If S_G is nullable then /*i. e., $\epsilon \in L(G)$
 Create in G' a new start symbol S^* .
 Add to $R_{G'}$ the two rules: $S^* \rightarrow \epsilon$ and $S^* \rightarrow S_G$.
3. Return G'' .

Example: $Bal = \{w \in \{(),\}^* : \text{the parentheses are balanced}\}$.



The new grammar built is better than the original one. The string $((()))$ has only one parse tree. But it is still ambiguous as the string $()()()$ has two parse trees?



Replace

$S \rightarrow SS$ with one of:

$S \rightarrow S S_1$ /* force branching to the left

$S \rightarrow S_1 S$ /* force branching to the right

So we get:

$S^* \rightarrow \epsilon | S$

$S \rightarrow S S_1$ /* force branching only to the left

$S \rightarrow S_1$ /* add rule

$S_1 \rightarrow (S) | ()$

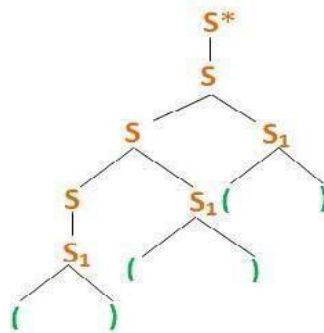
Unambiguous Grammar for Bal = $\{w \in \{(),\}^* : \text{the parentheses are balanced}\}$. $G = \{ \{S, (\}, \{), \{(), R, S \}$ where

$$S^* \rightarrow \varepsilon_n \mid S$$

$$S \rightarrow SS_1 \mid S_1 S_1$$

$$\rightarrow (S) \mid ()$$

The parse tree obtained for the string $()()()$ is



Unambiguous Arithmetic Expressions

Grammar is ambiguous in two ways:

a. It fails to specify associativity.

Ex: there are two parses for the string $\text{id} + \text{id} + \text{id}$, corresponding to the bracketing $(\text{id} + \text{id}) + \text{id}$ and $\text{id} + (\text{id} + \text{id})$

b. It fails to define a precedence hierarchy for the operations $+$ and $*$.

Ex: there are two parses for the string $\text{id} + \text{id} * \text{id}$, corresponding to the bracketing $(\text{id} + \text{id}) * \text{id}$ and $\text{id} + (\text{id} * \text{id})$

The unambiguous grammar for the arithmetic expression is:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F \mid T$$

$$\rightarrow F$$

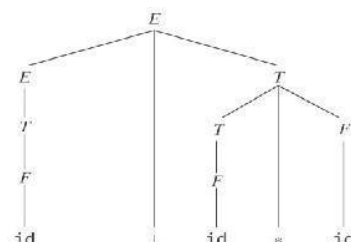
$$F \rightarrow (E)$$

$$F \rightarrow \text{id}$$

For identical operators: forced branching to go in a single direction (to the left). For

precedence Hierarchy: added the levels T (for term) and F (for factor)

The single parse tree obtained from the unambiguous grammar for the arithmetic expression is:



Proving that the grammar is Unambiguous

A grammar is unambiguous iff for all strings w , at every point in a leftmost derivation or rightmost derivation of w , only one rule in G can be applied.

$$S^* \rightarrow \varepsilon \quad \text{---(1)}$$

$$S^* \rightarrow S \quad \text{---(2)}$$

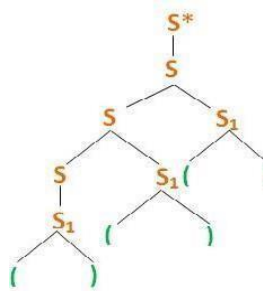
$$S \rightarrow SS_1 \quad \text{---(3)}$$

$$S \rightarrow S_1 \quad \text{---(4)}$$

$$S_1 \rightarrow (S) \quad \text{---(5)}$$

$$S_1 \rightarrow () \quad \text{---(6)}$$

$$S^* \Rightarrow S \Rightarrow SS_1 \Rightarrow SS_1S_1 \Rightarrow S_1S_1S_1 \Rightarrow ()S_1S_1 \Rightarrow ()()S_1 \Rightarrow ()()()$$



Inherent Ambiguity

In many cases, for an ambiguous grammar G , it is possible to construct a new grammar G' that generate $L(G)$ with less or no ambiguity. However, not always. Some languages have the property that every grammar for them is ambiguous. We call such languages inherently ambiguous.

Example: $L = \{a^i b^j c^k : i, j, k \geq 0, i=j \text{ or } j=k\}$.

Every string in L has either (or both) the same number of a 's and b 's or the same number of b 's and c 's. $L = \{a^n b^n c^m : n, m \geq 0\} \cup \{a^n b^m c^m : n, m \geq 0\}$.

One grammar for L has the rules: S

$$\rightarrow S_1 | S_2$$

$$S_1 \rightarrow S_1 c | A \quad /* \text{Generate all strings in } \{a^n b^n c^m\}. A$$

$$\rightarrow aAb | \varepsilon$$

$$S_2 \rightarrow aS_2 | B \quad /* \text{Generate all strings in } \{a^n b^m c^m\}. B$$

$$\rightarrow bBc | \varepsilon$$

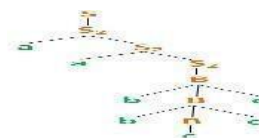
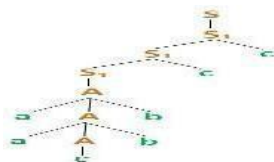
Consider the string $a^2 b^2 c^2$.

It has two distinct derivations, one through S_1 and the other through S_2 $S \Rightarrow$

$$S_1 \Rightarrow S_1 c \Rightarrow S_1 cc \Rightarrow Acc \Rightarrow aAbcc \Rightarrow aaAbbcc \Rightarrow aabbcc$$

$$S \Rightarrow S_2 \Rightarrow aS_2 \Rightarrow aaS_2 \Rightarrow aaB \Rightarrow aabBc \Rightarrow aabbBcc \Rightarrow aabbcc$$

Given any grammar G that generates L , there is at least one string with two derivations in G .



Both of the following problems are undecidable:

- Given a context-free grammar G , is G ambiguous?
- Given a context-free language L , is L inherently ambiguous?

8. Normal Forms

We have seen in some grammar where RHS is ϵ , it makes grammar harder to use. Let's see what happens if we carry the idea of getting rid of ϵ -productions a few steps farther. To make our tasks easier we define normal forms.

Normal Forms - When the grammar rules in G satisfy certain restrictions, then G is said to be in Normal Form.

- Normal Forms for queries & data can simplify database processing.
- Normal Forms for logical formulas can simplify automated reasoning in AI systems and in program verification systems.
- It might be easier to build a parser if we could make some assumptions about the form of the grammar rules that the parser will use.

Normal Forms for Grammar

Among several normal forms, two of them are:-

- Chomsky Normal Form (CNF)
- Greibach Normal Form (GNF)

Chomsky Normal Form (CNF)

In CNF we have restrictions on the length of RHS and the nature of symbols on the RHS of the grammar rules.

A context-free grammar $G = (V, \Sigma, R, S)$ is said to be in Chomsky Normal Form (CNF), iff every rule in R is of one of the following forms:

$$X \rightarrow a \quad \text{where } a \in \Sigma, \text{ or}$$

$$X \rightarrow BC \quad \text{where } B \text{ and } C \in V - \Sigma$$

Example: $S \rightarrow AB, A \rightarrow a, B \rightarrow b$

Every parse tree that is generated by a grammar in CNF has a branching factor of exactly 2 except at the branches that leads to the terminal nodes, where the branching factor is 1.

Using this property parser can exploit efficient data structure for storing and manipulating binary trees. Define straight forward decision procedure to determine whether w can be generated by a CNF grammar G . Easier to define other algorithms that manipulate grammars.

Greibach Normal Form (GNF)

GNF is a context free grammar $G = (V, \Sigma, R, S)$, where all rules have one of the following forms: $X \rightarrow a\beta$ where $a \in \Sigma$ and $\beta \in (V - \Sigma)^*$

Example: $S \rightarrow aA|aAB, A \rightarrow a, B \rightarrow b$

In every derivation precisely one terminal is generated for each rule application. This property is useful to define a straight forward decision procedure to determine whether w can be generated by GNF grammar G . GNF grammars can be easily converted to PDA with no ϵ transitions.

Converting to Chomsky Normal Form

Apply some transformation to G such that the language generated by G is unchanged.

1. Rule Substitution.

Example: $X \rightarrow aYc$ $Y \rightarrow bY$ $Y \rightarrow ZZ$ equivalent grammar constructed is $X \rightarrow abc$ $aZZc$

There exists 4-steps algorithm to convert a CFG G into a new grammar G_c such that: $L(G) = L(G_c) - \{\epsilon\}$

convertChomsky(G :CFG)=

1. $G' = \text{removeEps}(G:\text{CFG})$ $S \rightarrow \epsilon$
2. $G'' = \text{removeUnits}(G':\text{CFG})$ $A \rightarrow B$
3. $G''' = \text{removeMixed}(G'':\text{CFG})$ $A \rightarrow aB$
4. $G^{iv} = \text{removeLong}(G''':\text{CFG})$ $S \rightarrow ABCD$

return G_c

Remove Epsilon using removeEps(G :CFG)

Find the set N of nullable variables in G .

X is nullable iff either $X \rightarrow \epsilon$ or $(X \rightarrow A, A \rightarrow \epsilon): X \rightarrow \epsilon$

Example 1: $G: S \rightarrow aACa$

$A \rightarrow B|a$

$B \rightarrow C|c$

$C \rightarrow cC|\epsilon$

Now, since $C \rightarrow \epsilon$, C is nullable

since $B \rightarrow C$, B is nullable

since $A \rightarrow B$, A is nullable

Therefore $N = \{A, B, C\}$

removeEps returns G' :

$S \rightarrow aACa|aAa|aCa|aa$

$A \rightarrow B|a$

$B \rightarrow C|c$

$C \rightarrow cC|c$

Remove Unit Productions using removeUnits(G :CFG)

Unit production is a rule whose right hand side consists of a single nonterminal symbol. Ex: $A \rightarrow B$. Remove all unit production from G' .

Consider the remaining rules of G' .

G' : $S \rightarrow aACa|aAa|aCa|aa$

$A \rightarrow B|a$

$B \rightarrow C|c$

$C \rightarrow cC|c$

Remove $A \rightarrow B$ But $B \rightarrow C|c$, so Add $A \rightarrow C|c$

Remove $B \rightarrow C$ Add $B \rightarrow cC$ ($B \rightarrow c$, already there)

Remove $A \rightarrow C$ Add $A \rightarrow cC$ ($A \rightarrow c$, already there)

removeUnitsreturnsG":

$$S \rightarrow aACa | aAa | aCa | aa$$

$$A \rightarrow cC | a | c$$

$$B \rightarrow cC | c$$

$$C \rightarrow cC | c$$

RemoveMixedusingremoveMixed(G":CFG)

Mixed is a rule whose right hand side consists of combination of terminals or terminals with nonterminalsymbol. Create a new nonterminal T_a for each terminal $a \in \Sigma$. For each T_a , add the rule $T_a \rightarrow a$

Consider the remaining rules of G":

$$S \rightarrow aACa | aAa | aCa | aa$$

$$A \rightarrow cC | a | c$$

$$B \rightarrow cC | c$$

$$C \rightarrow cC | c$$

removeMixedreturnsG":

$$S \rightarrow T_aACT_a | T_aAT_a | T_aCT_a | T_aT_a$$

$$A \rightarrow T_cC | a | c$$

$$B \rightarrow T_cC | c$$

$$C \rightarrow T_cC | c$$

$$T_a \rightarrow a$$

$$T_c \rightarrow c$$

RemoveLongusingremoveLong(G":CFG)

Long is a rule whose right hand side consists of more than two nonterminalsymbol. R: $A \rightarrow$

BCDE is replaced as: $A \rightarrow BM_2$

$$M_2 \rightarrow CM_3$$

$$M_3 \rightarrow DE$$

Consider the remaining rules of G": S

$$\rightarrow T_aACT_a | T_aAT_a | T_aCT_a$$

Now, by applying removeLong we get: S

$$\rightarrow T_aS_1$$

$$S_1 \rightarrow AS_2$$

$$S_2 \rightarrow CT_aS$$

$$\rightarrow T_aS_3S_3$$

$$\rightarrow AT_aS$$

$$\rightarrow T_aS_2$$

Now, by apply removeLong returns G^V :

$$S \rightarrow T_a S_1 \mid T_a S_3 \mid T_a S_2 \mid T_a T_a S_1 \rightarrow$$

$$A S_2$$

$$S_2 \rightarrow C T_a S_3$$

$$\rightarrow A T_a$$

$$A \rightarrow T_c C \mid a \mid c$$

$$B \rightarrow T_c C \mid c$$

$$C \rightarrow T_c C \mid c$$

$$T_a \rightarrow a$$

$$T_c \rightarrow c$$

Example 2: Apply the normalizationalgorithm to convert the grammar to CNF $G: S$

$$\rightarrow aSa \mid B$$

$$B \rightarrow bbC \mid bb$$

$$C \rightarrow cC \mid \epsilon$$

removeEps($G:CFG$) returns

$$G': S \rightarrow aSa \mid B$$

$$B \rightarrow bbC \mid bb$$

$$C \rightarrow cC \mid c$$

removeUnits($G':CFG$) returns

$$G'': S \rightarrow aSa \mid bbC \mid bb$$

$$B \rightarrow bbC \mid bb$$

$$C \rightarrow cC \mid c$$

removeMixed($G'':CFG$) returns

$$G''': S \rightarrow T_a S T_a \mid T_b T_b C \mid T_b T_b$$

$$B \rightarrow T_b T_b C \mid T_b T_b$$

$$C \rightarrow T_c C \mid c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

removeLong($G''' :CFG$) returns

$$G^V: S \rightarrow T_a S_1 \mid T_b S_2 \mid T_b T_b$$

$$S_1 \rightarrow S T_a$$

$$S_2 \rightarrow T_b C$$

$$B \rightarrow T_b S_2 \mid T_b T_b$$

$$C \rightarrow T_c C \mid c \quad T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

Example3: Apply the normalizationalgorithm to convert the grammar to CNF G:

$$\begin{aligned} S &\rightarrow ABC \\ A &\rightarrow aC|D \\ B &\rightarrow bB|\varepsilon|A \\ C &\rightarrow Ac|\varepsilon|Cc \\ D &\rightarrow aa \end{aligned}$$

removeEps(G:CFG) returns G':

$$\begin{aligned} S &\rightarrow ABC|AC|AB|A \\ A &\rightarrow aC|D|a \\ B &\rightarrow bB|A|b \\ C &\rightarrow Ac|Cc|c \\ D &\rightarrow aa \end{aligned}$$

removeUnits(G':CFG) returns

G'':
$$\begin{aligned} S &\rightarrow ABC|AC|AB|aC|aa|a \\ A &\rightarrow aC|aa|a \\ B &\rightarrow bB|aC|aa|a|b \\ C &\rightarrow Ac|Cc|c \\ D &\rightarrow aa \end{aligned}$$

removeMixed(G'':CFG) returns

G''':
$$\begin{aligned} S &\rightarrow ABC|AC|AB|T_aC|T_aT_a|a \\ A &\rightarrow T_aC|T_aT_a|a \\ B &\rightarrow T_bB|T_aC|T_aT_a|a|b \\ C &\rightarrow AT_c|CT_c|c \\ D &\rightarrow T_aT_aT_a \\ &\rightarrow aT_b \rightarrow \\ &bT_c \rightarrow c \end{aligned}$$

removeLong(G''':CFG) returns

G'':
$$\begin{aligned} S &\rightarrow AS_1|AC|AB|T_aC|T_aT_a|aS_1 \rightarrow \\ &BC \\ A &\rightarrow T_aC|T_aT_a|a \\ B &\rightarrow T_bB|T_aC|T_aT_a|a|b \\ C &\rightarrow AT_c|CT_c|c \\ D &\rightarrow T_aT_aT_a \\ &\rightarrow aT_b \rightarrow \\ &bT_c \rightarrow c \end{aligned}$$

9. Pushdown Automata

An acceptor for every context-free language. A pushdown automata, or PDA, is a finite state machine that has been augmented by a single stack.

Definition of a (NPDA) Pushdown Automaton M

$= (K, S, G, \Delta, s, A)$, where:

- K is a finite set of states, S is the input alphabet,
- G is the stack alphabet,
- $s \in K$ is the initial state,
- $A \subseteq K$ is the set of accepting states, and Δ is the transition relation.

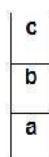
Δ is the transition relation. It is a finite subset of

$$\underbrace{(K \times (\Sigma \cup \{\epsilon\}) \times \Gamma^*)}_{\substack{\text{state} \quad \text{input or } \epsilon \quad \text{string of} \\ \text{symbols} \\ \text{to pop} \\ \text{from top} \\ \text{of stack}}} \times \underbrace{(K \times \Gamma^*)}_{\substack{\text{state} \quad \text{string of} \\ \text{symbols} \\ \text{to push} \\ \text{on top} \\ \text{of stack}}}$$

Configuration

A configuration of PDA M is an element of $K \times S^* \times G^*$. Current state, Input that is still left to read and, Contents of its stack.

The initial configuration of a PDA M , on input w , is (s, w, ϵ) .



will be written as cba



If $s_1 s_2 \dots s_n$ is pushed onto the stack: the value after the push is $s_1 s_2 \dots s_n c b a$

Yields-in-one-step written $\vdash_M$ relates configuration 1 to

A computation by M is a finite sequence of configurations $C_0, C_1, C_2, \dots, C_n$ for some $n \geq 0$ such that:

- C_0 is an initial configuration
- C_n is of the form (q, ε, γ) , for some $q \in K$ and some string γ in G^* , and
- $C_0 \vdash_M C_1 \vdash_M C_2 \vdash_M \dots \vdash_M C_n$.

Nondeterminism

If M is in some configuration (q_1, s, γ) it is possible that:

- Δ contains exactly one transition that matches. In that case, M makes the specified move.
- Δ contains more than one transition that matches. In that case, M chooses one of them.
- Δ contains no transition that matches. In that case, the computation that led to that configuration halts.

Accepting

Let C be a computation of M on input w . C is an accepting configuration

$$\text{iff } C = (s, w, \gamma) \vdash_M (q, \varepsilon, \gamma), \text{ for } q \in A.$$

A computation accepts only if it runs out on input w in an accepting state and the stack is empty.

C is a rejecting configuration iff $C = (s, w, \gamma) \vdash_M (q, w', \alpha)$,

where C is not an accepting computation and where M has no move that it can make from (q, w', α) .

A computation can reject only if the criteria for acceptance have not been met and there are no further moves that can be taken.

Let w be a string that is an element of S^* . Then:

- M accepts w if at least one of its computations accepts.
- M rejects w if all of its computations reject.

The language accepted by M , denoted $L(M)$, is the set of all strings accepted by M . M rejects a string w iff all paths reject it.

It is possible that, on input w , M neither accepts nor rejects. In that case, no path accepts and some path does not reject.

Transition

Transition $(q_1, c, 1, \gamma_1, \gamma_2)$ "If c matches the current top of the stack, then the transition can be taken. When taken, c will be removed from the input, 1 will be pushed onto the stack, and 2 will be pushed onto the stack. M cannot peek at the top of the stack without the stack with

- If $c = \varepsilon$, the transition can be taken without consuming any input.
- If $\gamma_1 = \varepsilon$, the transition can be taken without checking the stack or popping anything. Note: it's not "the stack is empty".
- If $\gamma_2 = \varepsilon$, no is pushed onto the stack when the transition is taken.

Example2ofRejecting--Input string=((()))

Transition	State	Unreadinput	Stack
	S	((()))	ϵ
1	S	((()))	(
1	S	((()))	((
1	S	((()))	((()
2	S	((()))	((()
2	S	((()))	((()

$(S, ((())), \epsilon) \rightarrow (S, ((()), (| - (S, ()), ((| - (S, ()), ((| - (S, ()), ((| - (S, ()), ((| - (S, \epsilon, (|$

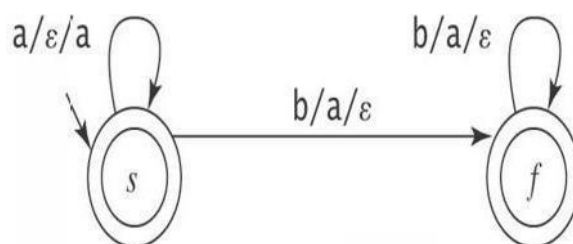
The computation has reached the final state S and runs out of input, but still the string is rejected because the stack is not empty.

Example2: APDA for $A^n B^n = \{a^n b^n : n \geq 0\}$ $M =$

(K, S, G, Δ, s, A) ,

where:

$K = \{s, f\}$ the states
 $S = \{a, b\}$ the input alphabet
 $\Gamma = \{a\}$ the stack alphabet
 $A = \{s, f\}$ the accepting state
 $\Delta = \{(s, a, \epsilon), (s, a)\} \quad \text{---- (1)}$
 $\{(s, b, a), (f, \epsilon)\} \quad \text{--- (2)}$
 $\{(f, b, a), (f, \epsilon)\} \quad \text{---- (3)}$



An Example of Accepting-- Input string= aabb

$(f, aabb, \epsilon) \rightarrow (f, abb, a) \rightarrow (f, bb, aa) \rightarrow (f, b, a) \rightarrow (f, \epsilon, \epsilon)$

The computation has reached the final state f, the input string is consumed and the stack is empty. Hence the string aabb is accepted.

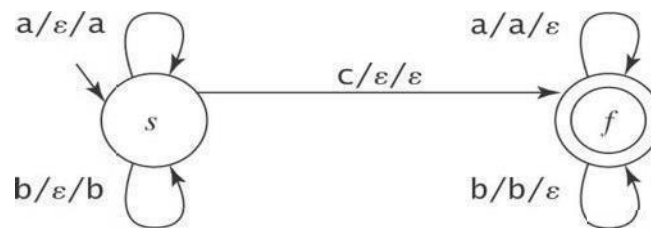
Example3: APDA for $\{wcw^R : w \in \{a, b\}^*\}$ $M =$

(K, S, G, Δ, s, A) ,

where:

$K = \{s, f\}$ the states
 $S = \{a, b, c\}$ the input alphabet
 $\Gamma = \{a, b\}$ the stack alphabet
 $A = \{f\}$ the accepting state

- $\Delta = \{((s, a, \epsilon), (s, a)) \text{ ---- (1)}$
 $((s, b, \epsilon), (s, b)) \text{ ---- (2)}$
 $((s, c, \epsilon), (f, \epsilon)) \text{ ---- (3)}$
 $((f, a, a), (f, \epsilon)) \text{ ---- (4)}$
 $((f, b, b), (f, \epsilon)) \}$



An Example of Accepting -- Input string = abcba

$(s, abcba, \epsilon) \xrightarrow{a/\epsilon/a} (s, bcba, a) \xrightarrow{b/\epsilon/b} (s, cba, ba) \xrightarrow{c/\epsilon/\epsilon} (f, ba, ba) \xrightarrow{a/a/\epsilon} (f, ,) \xrightarrow{b/b/\epsilon} (f, ,) \xrightarrow{\epsilon/\epsilon} (f, ,)$

The computation has reached the final state f, the input string is consumed, and the stack is empty.

Hence the string abcba is accepted.

Example 4: APDA for $A^n B^{2n} = \{a^n b^{2n} : n \geq 0\}$ $M =$

(K, S, G, Δ, s, A) ,

where:

$K = \{s, f\}$ the states
 $S = \{a, b\}$ the input alphabet
 $\Gamma = \{a\}$ the stack alphabet
 $A = \{s, f\}$ the accepting state

- $\Delta = \{((s, a, \epsilon), (s, aa)) \text{ ----- (1)}$
 $((s, b, a), (f, \epsilon)) \text{ ---- (2)}$
 $((f, b, a), (f, \epsilon)) \}$ ---- (3)



An Example of Accepting -- Input string = aabbbb

$(s, aabbbb, \epsilon) \xrightarrow{a/\epsilon/aa} (s, abbbb, aa) \xrightarrow{b/a/\epsilon} (f, bbbb, aaa) \xrightarrow{b/a/\epsilon} (f, bbb, aa) \xrightarrow{b/a/\epsilon} (f, bb, a) \xrightarrow{b/a/\epsilon} (f, ,) \xrightarrow{\epsilon/\epsilon} (f, ,)$

10. Deterministic and Nondeterministic PDAs

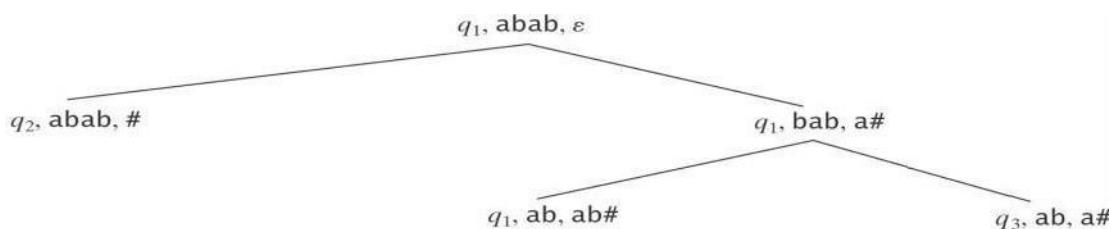
APDA is deterministic iff:

- Δ_M contains no pair of transitions that compete with each other, and
- whenever M is in an accepting configuration it has no available moves.
- If q is an accepting state of M , then there is no transition $((q, e, e), (p, a))$ for any p or a .

Unfortunately, unlike FSMs, there exist NDPDAs for which no equivalent DPDA exists.

Exploiting Nondeterministic

Previous examples are DPDA, where each machine followed only a single computational path. But many useful PDAs are not deterministic, where from a single configuration there exist multiple competing moves. As in FSMs, the easiest way to envision NDPDA the operation of a M is as a tree.



Each node in the tree corresponds to a configuration of M and each path from the root to a leaf node may correspond to one computation that M might perform. The state, the stack and the remaining input can be different along different paths. As a result, it will not be possible to simulate all paths in parallel, the way we did for NDFSMs.

Example 1: PDA for PalEven = $\{ww^R : w \in \{a, b\}^*\}$.

The set of even length palindromes of a 's and b 's = $\{\epsilon, aa, bb, aaaa, abba, baab, bbbb, \dots\}$

$M = (K, S, G, \Delta, s, A)$,

where:

$K = \{s, f\}$	the states
$S = \{a, b\}$	the input alphabet
$\Gamma = \{a, b\}$	the stack alphabet
$A = \{f\}$	the accepting state

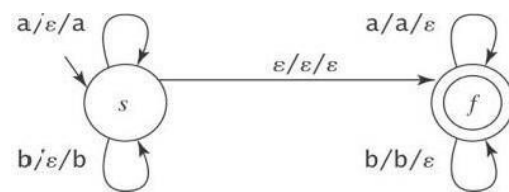
$\Delta = \{((s, a, \epsilon), (s, a)) \text{ ---- (1)}$

$((s, b, \epsilon), (s, b)) \text{ ---- (2)}$

$((s, \epsilon, \epsilon), (f, \epsilon)) \text{ ---- (3)}$

$((f, a, a), (f, \epsilon)) \text{ ---- (4)}$

$((f, b, b), (f, \epsilon)) \text{ ---- (5)}$



Example2: PDA for $\{w \in \{a, b\}^* : \#_a(w) = \#_b(w)\} = \text{EqualNumbers of a's and b's}$.

$L = \{\epsilon, ab, ba, abba, aabb, baba, bbaa, \dots\}$

$M = (K, S, G, \Delta, s, A)$,

where:

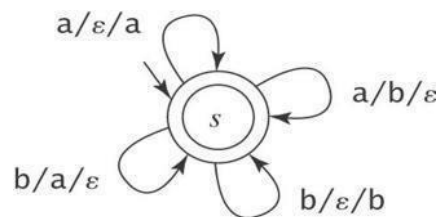
$K = \{s\}$ the states
 $S = \{a, b\}$ the input alphabet
 $\Gamma = \{a, b\}$ the stack alphabet
 $A = \{s\}$ the accepting state

$\Delta = \{((s, a, \epsilon), (s, a)) \text{ ---- (1)}$

$((s, b, \epsilon), (s, b)) \text{ ---- (2)}$

$((s, a, b), (s, \epsilon)) \text{ ---- (3)}$

$((s, b, a), (s, \epsilon)) \text{ ---- (4)}$

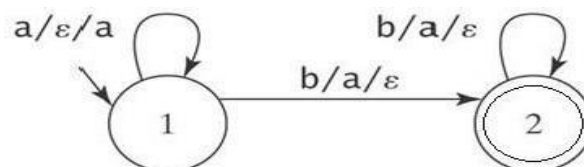


Example3: The a Region and the b Region are Different. $L = \{a^m b^n : m \neq n; m, n > 0\}$

It is hard to build a machine that looks for something negative, like $\neq$. But we can break L into two sublanguages: $\{a^m b^n : 0 < n < m\}$ and $\{a^m b^n : 0 < m < n\}$

- If stack and input are empty, halt and reject
- If input is empty but stack is not ($m > n$) (accept)
- If stack is empty but input is not ($m < n$) (accept) Start

with the case where $n = m$

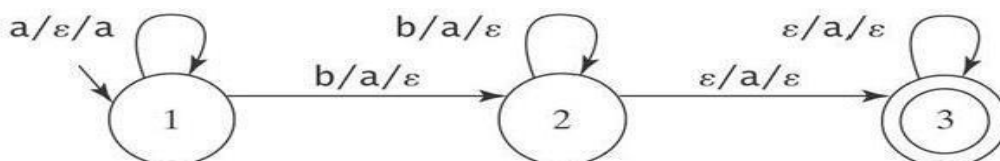


$\Delta = \{((1, a, \epsilon), (1, a)) \text{ ---- (1)}$

$((1, b, a), (2, \epsilon)) \text{ ---- (2)}$

$((2, b, a), (2, \epsilon)) \text{ ---- (3)}$

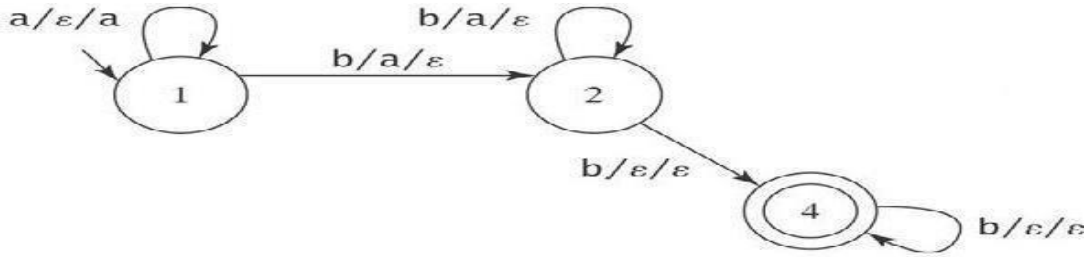
If input is empty but stack is not ($m > n$) (accept):



$\Delta = \{ ((1, a, \epsilon), (1, a)) \text{ ---- (1)}$
 $((1, b, a), (2, \epsilon)) \text{ ---- (2)}$
 $((2, b, a), (2, \epsilon)) \text{ ---- (3)}$
 $((2, \epsilon, a), (3, \epsilon)) \text{ ---- (4)}$

$((3, \epsilon, a), (3, \epsilon)) \text{ } \} \text{----- (5)}$

If stack is empty but input is not ($m < n$) (accept):



$\Delta = \{ ((1, a, \epsilon), (1, a)) \text{ ---- (1)}$
 $((1, b, a), (2, \epsilon)) \text{ ---- (2)}$
 $((2, b, a), (2, \epsilon)) \text{ ---- (3)}$
 $((2, b, \epsilon), (4, \epsilon)) \text{ ---- (4)}$

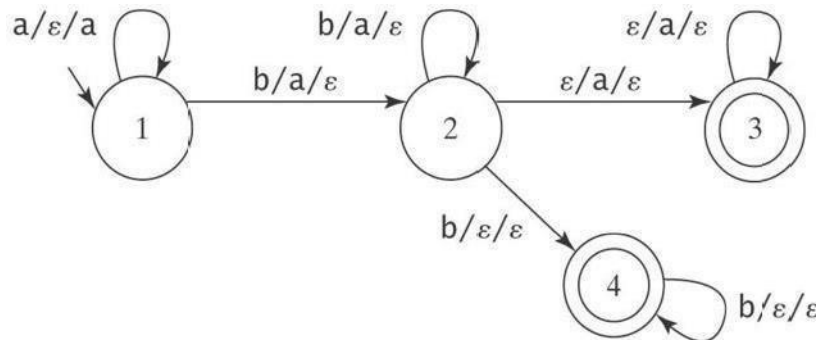
$((4, b, \epsilon), (4, \epsilon)) \text{ } \} \text{----- (5)}$

Putting all together the PDA obtained is $M =$

$(K, S, G, \Delta, s, A),$

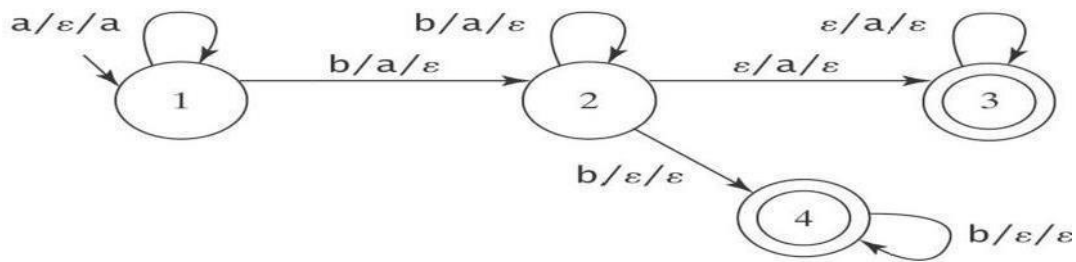
where:

$K = \{1, 2, 3, 4\}$ the states
 $S = \{a, b\}$ the input alphabet
 $\Gamma = \{a\}$ the stack alphabet
 $A = \{3, 4\}$ the accepting state



$\Delta = \{ ((1, a, \epsilon), (1, a)) \text{ ---- (1)}$
 $((1, b, a), (2, \epsilon)) \text{ ---- (2)}$
 $((2, b, a), (2, \epsilon)) \text{ ---- (3)}$
 $((2, \epsilon, a), (3, \epsilon)) \text{ ---- (4)}$
 $((3, \epsilon, a), (3, \epsilon)) \text{ } \} \text{---- (5)}$
 $((2, b, \epsilon), (4, \epsilon)) \text{ ---- (6)}$

$$((4, b, \varepsilon), (4, \varepsilon))\} \quad - \text{---}(7)$$



Two problems with this M:

1. We have no way to specify that a move can be taken only if the stack is empty.
2. We have no way to specify that the input stream is empty.
3. As a result, in most of its moves in state 2, M will have a choice of three paths to take.

Techniques for Reducing Nondeterminism

We saw nondeterminism arising from two very specific circumstances:

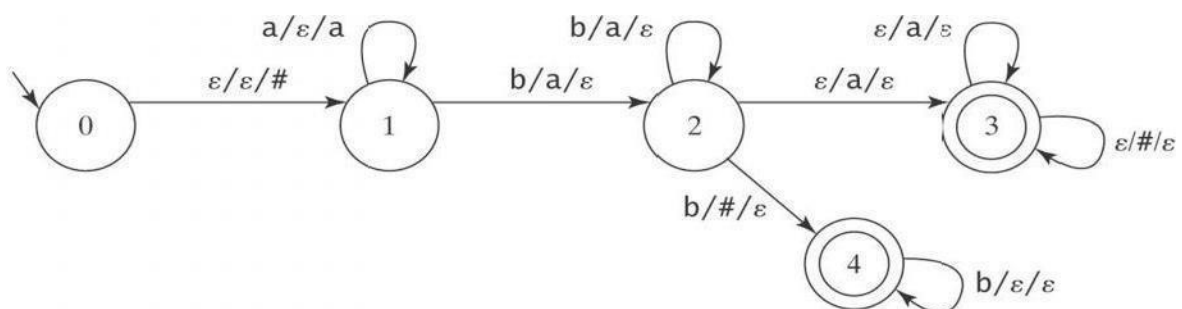
- A transition that should be taken only if the stack is empty competes against one or more moves that require a match of some string on the stack.
- A transition that should be taken only if the input stream is empty competes against one or more moves that require a match against a specific input character.

Case 1: A transition that should be taken only if the stack is empty competes against one or more moves that require a match of some string on the stack.

Problem: Nondeterminism could be eliminated if it were possible to check for an empty stack.

Solution: Using a special bottom-of-stack marker (#)

Before doing anything, push a special character onto the stack. The stack is then logically empty iff that special character (#) is at the top of the stack. Before M accepts a string, its stack must be completely empty, so the special character must be popped whenever M reaches an accepting state.



Now the transition back to state 2 no longer competes with the transition to state 4, which can only be taken when the # is the only symbol on the stack. The machine is still

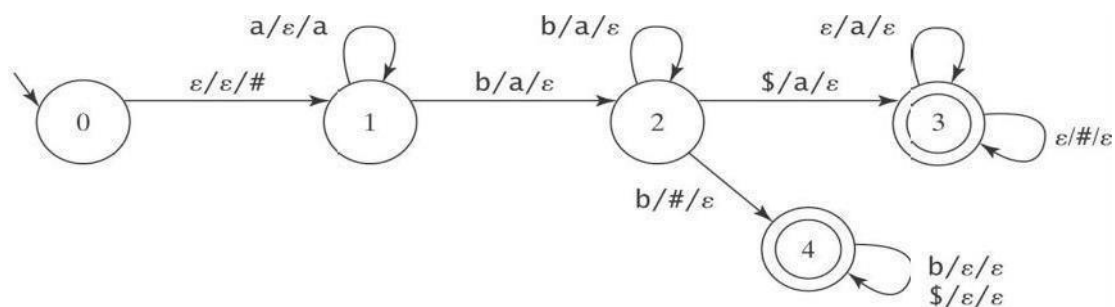
nondeterministic because the transition back to state 2 competes with the transition to state 3.

Case2: A transition that should be taken only if the input stream is empty competes against one or more moves that require a match against a specific input character.

Problem: Nondeterminism could be eliminated if it were possible to check for an empty input stream.

Solution: using a special end-of-string marker (\$)

Adding an end-of-string marker to the language to be accepted is a powerful tool for reducing nondeterminism. Instead of building a machine to accept a language L , build one to accept $L\$$, where $\$$ is a special end-of-string marker.



Now the transition back to state 2 no longer competes with the transition to state 3, since the can be taken when the \$ is read. The \$ must be read on all the paths, not just the one where we need it.

11. Nondeterminism and Halting

Recall Computation C of a PDA $M = (K, S, G, \Delta, s, A)$ on a string w is an accepting

computation iff $C = (s, w, \epsilon) \vdash_{M^*} (q, \epsilon, \epsilon)$, for some $q \in A$.

A computation C of M halts iff at least one of the following condition holds:

- C is an accepting computation, or
- C ends in a configuration from which there is no transition in Δ that can be taken.

M halts on w iff every computation of M on w halts. If M halts on w and does not accept, then we say that M rejects w .

For every CFL, we've proved that there exists a PDA M such that $L(M) = L$. Suppose that we would like to be able to:

1. Examine a string and decide whether or not it is in L .
2. Examine a string that is in L and create a parse tree for it.
3. Examine a string that is in L and create a parse tree for it in time that is linear in the length of the string.
4. Examine a string and decide whether or not it is in the complement of L .

For every regular language L , there exists a minimal deterministic FSM that accepts it. That minimal DFSA halts on all inputs, accepts all strings that are in L , and rejects all strings that are not in L .

But the facts about CFGs and PDAs are different from the facts about RLs and FSMs.

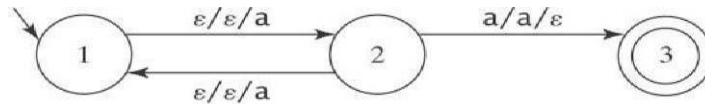
1. There are context-free languages for which no deterministic PDA exists.
2. It is possible that a PDA may
 - not halt,
 - not ever finish reading its input.

However, for an arbitrary PDA M , there exists M' that halts and $L(M') = L(M)$.

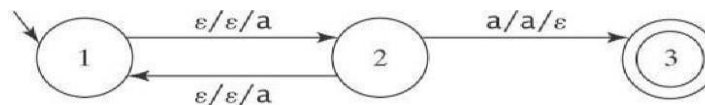
There exists no algorithm to minimize a PDA. It is undecidable whether a PDA is minimal.

Problem 2 : Let M be a PDA that accepts some language L . Then, on input w , if $w \in L$ then M will halt and accept. But if $w \notin L$ then, while M will not accept w , it is possible that it will not reject it either.

Example 1: Let $S = \{a\}$ and consider $M =$



For $L(M) = \{a\}$. The computation $(1, a, \epsilon) \rightarrow (2, a, a) \rightarrow (3, \epsilon, \epsilon)$ causes M to accept. Example 2: Consider $M =$



For $L(M) = \{aa\}$ or on any other input except a :

$(1, aa, \epsilon) \rightarrow (2, aa, a) \rightarrow (1, aa, aa) \rightarrow (2, aa, aaa) \rightarrow (1, aa, aaaa) \rightarrow (2, aa, aaaaa) \rightarrow \dots$ M will never halt because of one path never ends and none of the paths accepts.

The same problem with NDFSMs has a choice of two solutions.

- Converting NDFSMs to an equivalent DFSM using $ndfsm \rightarrow dfsm$ algorithm.
- Simulating NDFSMs using $ndfsm \rightarrow simulate$.

Neither of these approaches work on PDAs. There may not even be an equivalent deterministic PDA.

Solutions fall into two classes:

- Formal ones that do not restrict the class of the language that are being considered - converting grammar into normal forms like Chomsky or Greibach normal form.
- Practical ones that work only on a subclass of the CFLs - use grammars in natural forms.

12. Alternative Equivalent Definitions of a PDA

$PDA = (K, S, G, \Delta, s, A)$:

1. Allow M to pop and push any string in G^* .
2. M may pop only a single symbol but it may push any number of them.
3. M may pop and push only a single symbol.

M accepts its input w only if, when it finishes reading w , it is in an accepting state and its stack is empty.

There are two alternatives to this:

1. PDA by Final state: Accept if, when the input has been consumed, M lands in an accepting state, regardless of the contents of the stack.
2. PDA by Empty stack: Accept if, when the input has been consumed, the stack is empty, regardless of the state M is in.

All of these definitions are equivalent in the sense that, if some language L is accepted by a PDA using one definition, it can be accepted by some PDA using each of the other definitions. For example:- If some language L is accepted by a PDA by Final state then it can be accepted by PDA by Final state and empty stack. If some language L is accepted by a PDA by Final state and empty stack then can be accepted by PDA by Final state.

We can prove by showing algorithms that transform a PDA of one sort into an equivalent PDA of the other sort.

Equivalence

1. Given a PDA M that accepts by accepting state and empty stack, construct a new PDA M' that accepts by accepting state alone, where $L(M') = L(M)$.
2. Given a PDA M that accepts by accepting state alone, construct a new PDA M' that accepts by accepting state and empty stack, where $L(M') = L(M)$.

Hence we can prove that M' and M accept the same strings.

1. Accepting by Final state Alone

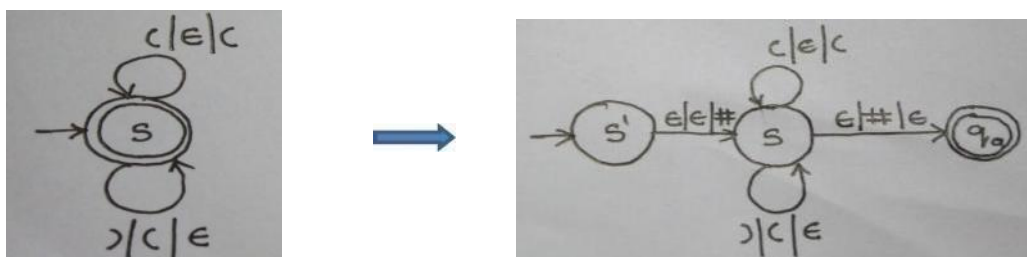
Define a PDA $M = (K, S, G, \Delta, s, A)$. Accepts when the input has been consumed, M lands in an accepting state, regardless of the contents of the stack. M accepts if $C = (s, w, \epsilon) \vdash_M^* (q, \epsilon, g)$, for some $q \in A$.

M' will have a single accepting state q_a . The only way for M' to get to q_a will be to land in an accepting state of M when the stack is logically empty. Since there is no way to check that the stack is empty, M' will begin by pushing a bottom-of-stack marker $\#$, onto the stack.

Whenever $\#$ is the top symbol of the stack, then the stack is logically empty. The construction proceeds as follows:

1. Initially, let $M' = M$.
2. Create new start states'.
Add the transition $((s', \epsilon, \epsilon), (s, \#))$,
3. For each accepting state a in M do:
Add the transition $((a, \epsilon, \#), (q_a, \epsilon))$,
4. Make q_a the only accepting state in M'

Example:



It is easy to see that M' lands in its accepting state (q_a) iff M lands in some accepting state with an empty stack. Thus M' and M accept the same strings.

2. Accepting by Final state and Empty stack

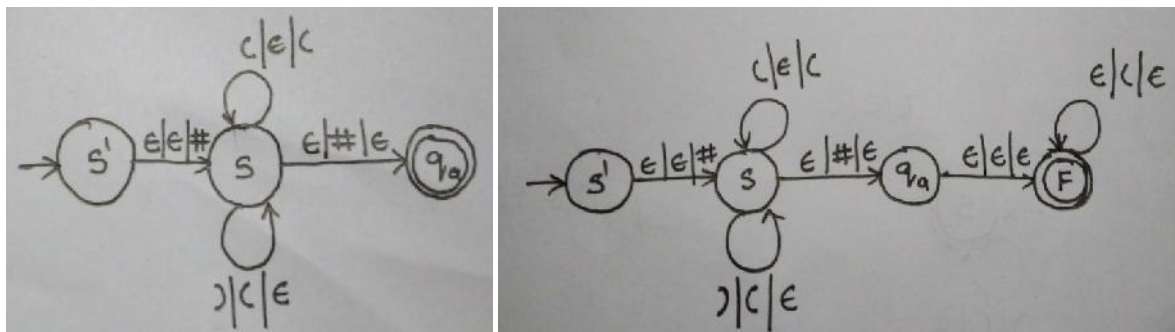
The construction proceeds as follows:

1. Initially, let $M' = M$.
2. Create a new accepting state F
3. From each accepting state a in M do:
Add the transition $((a, \epsilon, \epsilon), (F, \epsilon))$,
4. Make F the only accepting state in M'
5. For every element g of Γ ,
Add the transition to $M'((F, \epsilon, g), (F, \epsilon))$.

In other words, iff M accepts, go to the only accepting state of M' and clear the stack.

Thus M' will accept by accepting state and empty stack iff M accepts by accepting state.

Example:-



Thus M' and M accept the same strings.

13. Alternatives that are not equivalent to the PDA

We defined a PDA to be a finite state machine to which we add a single stack.

Two variants of that definition, each of which turns out to define a more powerful class of a machine.

1. First variant: add a first-in, first-out (FIFO) queue in place of a stack. Such machines are called tag systems or Post machines.
2. Second variant: add two stacks instead of one. The resulting machines are equivalent in computational power to Turing Machines.

Sl.No	SampleQuestions
1.	Define context free grammars and languages.
2.	Show a context-free grammar for each of the following languages L: a) $BalDelim = \{w : \text{where } w \text{ is a string of delimiters } (, , [,] , \{ , \} , \text{ that are properly balanced}\}.$ b) $\{a^i b^j : 2i = 3j + 1\}.$ c) $\{a^i b^j : 2i \neq 3j + 1\}.$ d) $\{a^i b^j c^k : i, j, k \geq 0 \text{ and } (i \neq j \text{ or } j \neq k)\}.$
3.	Define CFG. Design CFG for the language $L = \{a^n b^m : n \neq m\}$
4.	Apply the simplification algorithm to simplify the given grammar $S \rightarrow AB AC \quad A \rightarrow aAb \epsilon \quad B \rightarrow bAC \rightarrow bCa \quad D \rightarrow AB$
5.	Prove the correctness of the grammar for the language: $L = \{w \in \{a, b\}^* : \#_a(w) = \#_b(w)\}.$
6.	Define leftmost derivation and rightmost derivation. Given the following CFG. $E \rightarrow E + T T \quad T \rightarrow T * F F \quad F \rightarrow (E) a b c$ Draw parse tree for the following sentences and also derive the leftmost and rightmost derivations i) $(a+b)^*c$ ii) $(a)+b^*c$
7.	Consider the following grammar $G: S \rightarrow 0S1 SS 10$ Show a parse tree produced by G for each of the following strings: a) 010110 b) 00101101
8.	Define ambiguous and explain inherently ambiguous grammars.
9.	Prove whether the given grammar is ambiguous grammar or not. $E \rightarrow E + E \quad E \rightarrow E * E a b c$
10.	Prove that the following CFG is ambiguous $S \rightarrow iCtS iCtSeS x \quad C \rightarrow y$ for the string $iytytxex$
11.	Define Chomsky normal form. Apply the normalization algorithm to convert the grammar to Chomsky normal form. a. $S \rightarrow aSaS \rightarrow B \quad B \rightarrow bbC$ $B \rightarrow bb \quad C \rightarrow \epsilon \quad C \rightarrow cC$ b. $S \rightarrow ABC \quad A \rightarrow aC D \quad B \rightarrow bB \epsilon A C$ $\rightarrow Ac \epsilon Cc \quad D \rightarrow aa$
12.	Define Pushdown automata (NPDA). Design a NPDA for the CFG given in Question (2).
13.	Design a PDA for the given language L , where $L = \{w \in \{a, b\}^* : \#_a(w) = \#_b(w)\}.$
14.	Design a PDA for the language $L = \{a^i b^j c^k : i + j = k, i \geq 0, j \geq 0\}$
15.	Design a PDA for the language $L = \{a^n b^{2n} : n \geq 1\}$
16.	Design a PDA for the language $L = \{a^i b^j c^k : i + k = j, i \geq 0, k \geq 0\}$
17.	Design a PDA for the language $L = \{a^i b^j c^k : k + j = i, k \geq 0, j \geq 0\}$

